

2. Multimodal Visual Understanding (Text Version)

1. Course Content

1. Learn to use the robot's visual understanding function.
2. Learn the newly appearing key source code.

2. Preparation

2.1 Content Description

The virtual machine used by default with this product starts the Dify docker environment automatically at boot, so no additional startup is needed.

⚠ The same test instructions will not produce exactly the same reply content from the large model, and will differ slightly from the screenshots in the tutorial. If you need to enhance or reduce the diversity of the large model's replies, refer to **【AI Large Model Development】 - 【RAG Knowledge Base Deployment】** to modify the knowledge base and sample library.

Virtual machine terminal, manually start the Dify environment script:

```
sh ~/bringup_dify.sh
```

```
yahboom@yahboom:~$ sh ~/bringup_dify.sh
[+] up 13/13
✓ Container docker-sandbox-1      Running      0.0s
✓ Container docker-web-1          Running      0.0s
✓ Container docker-db_postgres-1  Healthy     22.5s
✓ Container docker-ssrf_proxy-1   Running      0.0s
✓ Container docker-redis-1        Running      0.0s
✓ Container docker-weaviate-1     Running      0.0s
✓ Container docker-worker-1       Running      0.0s
✓ Container docker-api_websocket-1 Running      0.0s
✓ Container docker-plugin_daemon-1 Running      0.0s
✓ Container docker-worker_beat-1  Running      0.0s
✓ Container docker-api-1          Running      0.0s
✓ Container docker-nginx-1        Running      0.0s
✓ Container docker-init_permissions-1 Exited       28.4s
yahboom@yahboom:~$
```

3. Running the Case

3.1 Starting the Program

Open the virtual machine terminal and enter the command to start the underlying proxy + camera topic:

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-23-21-30-23-024975-yahboom-608172
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [608193]
[INFO] [camera_driver-2]: process started with pid [608195]
[INFO] [robot_state_publisher-3]: process started with pid [608197]
[INFO] [joint_state_publisher-4]: process started with pid [608199]
[INFO] [ekf_node-5]: process started with pid [608201]
[micro_ros_agent-1] [1784813423.243996] info      | UDPv4AgentLinux.cpp | init
                        | running...          | port: 8090
[robot_state_publisher-3] [INFO] [1784813423.323961815] [robot_state_publisher]:
got segment base_footprint
[robot_state_publisher-3] [INFO] [1784813423.324159300] [robot_state_publisher]:
got segment base_link
[robot_state_publisher-3] [INFO] [1784813423.324177934] [robot_state_publisher]:
got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1784813423.324185448] [robot_state_publisher]:
got segment cam1_Link
[robot_state_publisher-3] [INFO] [1784813423.324192058] [robot_state_publisher]:
got segment cam2_Link
[robot_state_publisher-3] [INFO] [1784813423.324198417] [robot_state_publisher]:
```

Open another virtual machine terminal and enter the command:

```
ros2 launch multi_brains llm_agent_control.launch.py
```

After initialization is complete, the following content will be displayed:

```
G_MODE=False, LANGUAGE=en, CAR_TYPE=ptz
[llm_agent_node-1] [INFO] [1787128219.506352762] [llm_agent_node]: Initializing
LLM Agent Node...
[llm_agent_node-1] [INFO] [1787128219.515646094] [llm_agent_node]: [CONFIG] cmd_
vel_topic=/cmd_vel
[llm_agent_node-1] [INFO] [1787128219.522266585] [llm_agent_node]: [CONFIG] imag
e_cache_path=/home/yahboom/microros_ws/cache/llm_camera.jpg
[llm_agent_node-1] [INFO] [1787128219.523535816] [llm_agent_node]: [PTZ] startup
initialization enabled: x=0.0°, y=-45.0°, count=20, interval=0.10s
[llm_agent_node-1] [INFO] [1787128219.550460204] [llm_agent_node]: [FOLLOW] scan
_topic=/scan, obstacle_guard=True, buzzer_topic=/beep, buzzer=True
[llm_agent_node-1] [INFO] [1787128219.551155699] [llm_agent_node]: [DIFY] Initia
lize client: base_url=http://localhost/v1, api_key=app-SAnX****Zt2E
[llm_agent_node-1] [INFO] [1787128219.553635739] [llm_agent_node]: [DIFY] Testin
g connection...
[llm_agent_node-1] [INFO] [1787128219.569125658] [llm_agent_node]: [TRACK_KCF] S
ubscribed /tracking_bbox, type=smart_tracker/msg/TrackBox
[llm_agent_node-1] [INFO] [1787128219.569798915] [llm_agent_node]: LLM Agent Nod
e initialized successfully
[llm_agent_node-1] [INFO] [1787128219.585362718] [llm_agent_node]: [DIFY] Connec
tion successful ✓
[llm_agent_node-1] [INFO] [1787128221.526141357] [llm_agent_node]: [PTZ] Servo i
nitialization complete: x=0.0°, y=-45.0°
```

Then open another terminal and start:

```
ros2 run multi_brains terminal_client
```

```
yahboom@VM: ~  
=====   
> MicroROS LLM Agent Terminal Client   
=====   
> Enter natural-language commands to control the robot   
>   
> Examples:   
>   > Go to point A   
>   > Follow the red line   
>   > Look at what is ahead   
>   > Move forward 2 meters   
>   
> Commands:   
>   reset - Reset conversation context   
>   clear  - Clear screen   
>   help   - Show this help   
>   exit   - Exit program   
> =====
```

3.2 Test Cases

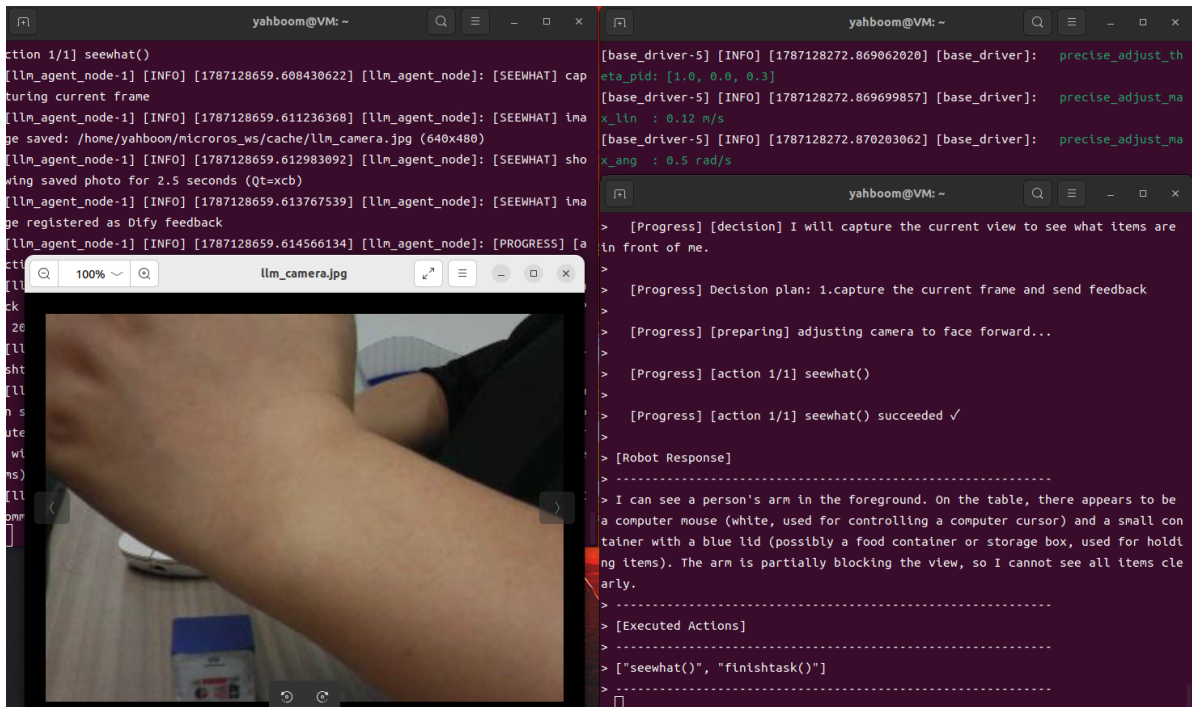
Here are two reference test cases. Users can create their own test instructions.

- Help me see what items are in front of you, and tell me what these items are used for.
- Please look to see if there is a blue block and a pack of tissues in front of you. If so, nod your head; if not, shake your head.

⚠ Do not add a period or any character at the end of the input text!

3.2.1 Case 1

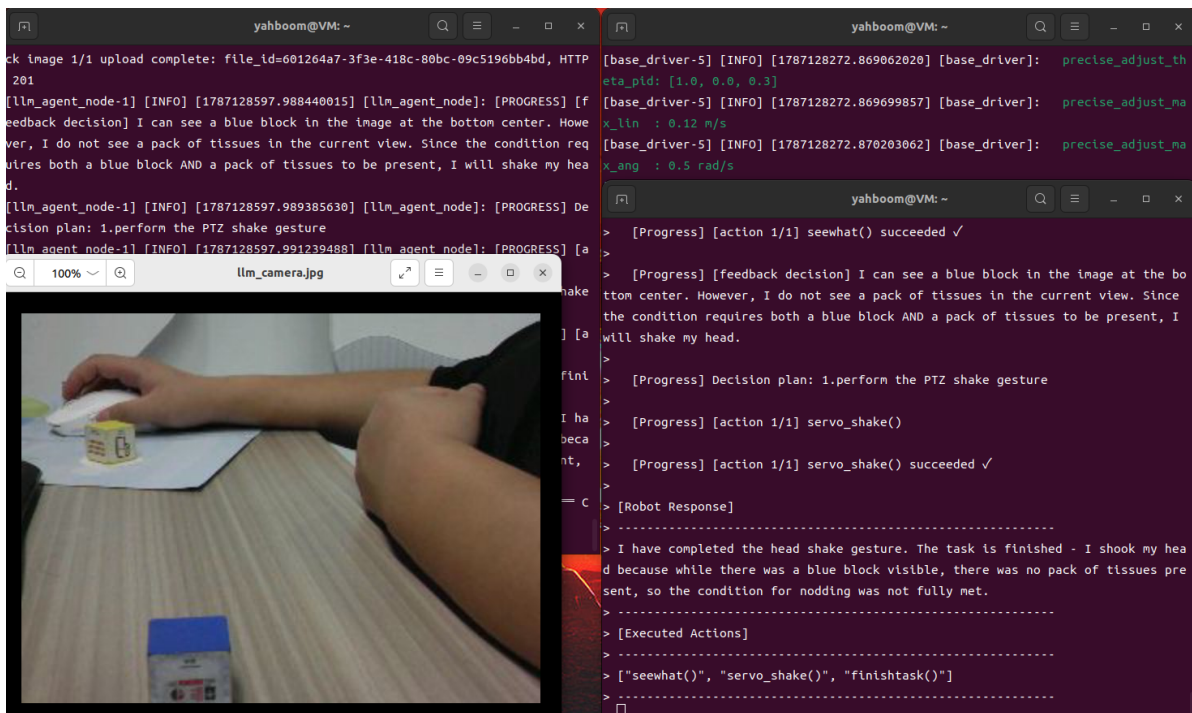
Enter the test case in the text interaction terminal. After waiting for the model to think, the model replies to the user and executes actions according to the user's instructions.



3.2.2 Case 2

The method is the same as Case 1. Enter Case 2 in the terminal, and the model will reply and execute actions according to the instructions.

Case 2 uses the pan-tilt servo camera version as an example. Nodding and shaking the head can also be replaced with commands such as the car moving forward or moving backward.



4. Source Code Analysis

Visual understanding is completed uniformly by `llm_agent_node.py`: the node subscribes to the camera topic `/camera/image_raw` and caches the latest image; when the execution layer outputs `seewhat()`, the node saves the current frame as an image and uploads it to Dify as a feedback attachment, and Dify then continues the next round of reasoning based on the image.

Current relevant source code paths:

```
~/microros_ws/src/multi_brains/multi_brains/llm_agent_node.py
~/microros_ws/src/multi_brains/multi_brains/services/dify_coordinator.py
~/microros_ws/src/multi_brains/multi_brains/controllers/ptz_controller.py
```

Core flow:

```
User inputs a vision question in terminal_client
-> Execution layer outputs seewhat()
-> llm_agent_node saves the current image from /camera/image_raw
-> Uploads the image through DifyCoordinator and feeds back "robot image
feedback completed"
-> Dify generates a reply based on the image or continues to output actions
-> terminal_client displays the final reply and action list
```

If the user asks to "look around", the execution layer will output `observe_surroundings()`, and the current node will control the pan-tilt to take photos of the front, right, and left in sequence, and feed multiple images to Dify at once.